



WEEK 2 - PYTHON PROGRAMMING

Operators, Strings & Conditionals

Expressions, text processing and decision making

Theory 2 h - Practice 2 h

Instructor: MinGi Kyung

Today's Agenda

- Part 1: Theory (2 h)
- Arithmetic, comparison and logical operators
- String indexing, slicing and methods
- Decision making with if, elif and else

Part 2: Practice (2 h)

Lab 1: String manipulation

Lab 2: Grade classifier

Lab 3: Rock-paper-scissors

Lab 4: Leap-year calculator

Four short Python programs



PART 1 - THEORY

Operators, Strings & Conditionals

Calculate with operators and compare values

Extract and transform text

Choose a branch based on a condition

Review: Input, Types & Output

- `input()` returns a string, even for numeric input.
- Convert numeric text with `int()` or `float()`.
- An expression combines values and operators.
- Predict the output before running the code.

```
a = "12"
b = "3"
print(a + b)           # 123
print(int(a) + int(b)) # 15

price = 12.5
quantity = 3
total = price * quantity
print(f"Total: {total:.2f}")
# Total: 37.50
```

Arithmetic Operators

- + and - add and subtract. * multiplies.
- / performs division. // floors the quotient.
- % gives the remainder. ** raises a power.
- Floor division rounds down: -7 // 3 is -3.
- Division or remainder by zero raises an error.

```
a, b = 7, 3
print(a + b)    # 10
print(a - b)    # 4
print(a * b)    # 21
print(a / b)    # 2.333...
print(a // b)   # 2
print(a % b)    # 1
print(a ** b)   # 343

minutes = 135
print(minutes // 60) # 2 hours
print(minutes % 60)  # 15 minutes
```

Comparison Operators

- `==` tests equality. `!=` tests inequality.
- `<`, `<=`, `>` and `>=` compare values.
- Comparisons return `True` or `False`.
- `=` assigns a value. `==` compares values.
- A chained comparison can check a range.

```
score = 85
print(score == 85) # True
print(score != 85) # False
print(score >= 60) # True
print(score < 60)  # False
print(0 <= score <= 100) # True

print("85" == 85)  # False
# Convert input before comparing.
```

Logical Operators

- and requires both conditions to be true.
- or requires at least one condition to be true.
- not reverses the truth value.
- Python short-circuits: it skips an operand when the result is already determined.

```
age = 20
has_id = True
print(age >= 18 and has_id) # True
print(age < 18 or has_id)   # True
print(not has_id)           # False

n = 0
print(n != 0 and 10 / n > 2)
# False: 10 / n is never evaluated.
```

Precedence & Assignment Updates

- `**` binds more tightly than `*` `/` `//` `%`, which bind more tightly than `+` `-`.
- Comparisons come before `not`, then `and`, then `or`.
- Parentheses make the intended grouping clear.
- `+=`, `-=` and `*=` update an existing variable.

```
print(2 + 3 * 4)      # 14
print((2 + 3) * 4)    # 20
print(-2 ** 2)        # -4
print((-2) ** 2)      # 4
```

```
score = 80
score += 5
print(score)          # 85
```

```
# Which output do you expect?
print(True or False and False)
# True
```

Strings & Indexing

- A string is a sequence of characters.
- The first character has index 0.
- Negative indices count from the end.
- `len(text)` returns the number of characters.
- An index outside the string raises `IndexError`.

```
text = "Python"
#      012345
print(len(text)) # 6
print(text[0])   # P
print(text[2])   # t
print(text[-1])  # n
print(text[-2])  # o

# text[6] raises IndexError
# ""[0] also raises IndexError
```

String Slicing

- `text[start:stop]` includes start and excludes stop.
- An omitted start begins at index 0. An omitted stop goes to the end.
- A third value sets the step.
- Slices tolerate bounds beyond the string.
- Strings are immutable. Create a new string to change the text.

```
text = "Python"
print(text[0:2]) # Py
print(text[2:])  # thon
print(text[:4])  # Pyth
print(text[::2]) # Pto
print(text[::-1]) # nohtyP
print(text[:99]) # Python

# text[0] = "J" is invalid.
text = "J" + text[1:]
print(text)      # Jython
```

String Methods & Membership

- `strip()` removes whitespace at both ends.
- `lower()` and `upper()` change letter case.
- `replace(old, new)` returns replaced text.
- `split()` separates text into a list of strings.
- `in` checks whether a substring occurs. Methods return results without changing the original string.

```
raw = "  Hello Python  "
clean = raw.strip()
print(clean)           # Hello Python
print(clean.lower())   # hello python
print(clean.upper())   # HELLO PYTHON
print(clean.replace("Python", "AI"))
# Hello AI
print(clean.split())
# ['Hello', 'Python']
print("Python" in clean) # True
print("python" in clean) # False
```

Joining Text & Formatting Values

- + concatenates strings. * repeats a string.
- str() converts a number to text.
- An f-string embeds expressions inside braces.
- .2f displays two digits after the decimal point.
- Normalize user input before comparing text.

```
name = "Mina"
score = 87.5
print("Hi, " + name) # Hi, Mina
print("-" * 5)       # -----
print("Score: " + str(score))
print(f"{name}: {score:.2f}")
# Mina: 87.50

answer = "  YES  "
print(answer.strip().lower() == "yes")
# True
```

Conditionals: if & else

- if runs an indented block when its condition is true.
- else runs when that condition is false.
- End the if and else headers with a colon.
- Use four spaces for each indentation level.
- Code after the block runs regardless of the branch.

```
age = 17
if age >= 18:
    print("Adult")
else:
    print("Under 18")
print("Check complete")
```

```
# Output:
# Under 18
# Check complete
```

Multiple Branches with elif

- elif checks another condition if earlier conditions were false.
- The first matching branch runs.
- An optional else handles all remaining cases.
- Check grade thresholds from highest to lowest.
- Separate if statements can run multiple blocks.

```
score = 85
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F"
print(grade) # B
```

Validation & Nested Conditions

- Validate a value before using it.
- A nested if runs inside another branch.
- Use a guard to avoid dividing by zero.
- For these labs, assume numeric input has the requested format.
- We will handle conversion errors with exceptions later.

```
a = float(input("First number: "))
b = float(input("Second number: "))

if b == 0:
    print("Cannot divide by zero")
else:
    result = a / b
    if result < 0:
        print("Negative result")
    print(f"Result: {result:.2f}")
```

Common Mistakes

- Use `==` for equality, rather than assignment `=`.
- Missing colons or indentation break a block.
- Convert text before comparing numeric values.
- `"False"` is a nonempty string, so `bool("False")` is `True`.
- Compare both alternatives explicitly when using `or`.

```
choice = "paper"

# Incorrect: "scissors" is truthy.
if choice == "rock" or "scissors":
    print("This always runs")

# Correct:
if (choice == "rock"
    or choice == "scissors"):
    print("Rock or scissors")
```



PART 2 - PRACTICE

Hands-on Labs

Lab 1: String manipulation (20 min)

Lab 2: Grade classifier (25 min)

Lab 3: Rock-paper-scissors (40 min)

Lab 4: Leap-year calculator (25 min)

Testing and submission (10 min)

Lab 1: String Manipulation

- Create strings.py and read a line of text.
- Remove surrounding whitespace.
- Print its length, uppercase form and reverse.
- Show its first character only when it is nonempty.
- Test a normal word, spaces only and a single character.

```
text = input("Text: ").strip()
print(f"Length: {len(text)}")
print(f"Upper: {text.upper()}")
print(f"Reverse: {text[::-1]}")
if text:
    print(f"First: {text[0]}")
else:
    print("No text entered")

# Input:   Python
# Length: 6, Upper: PYTHON
# Reverse: nohtyP, First: P
```

Lab 2: Grade Classifier

- Create grades.py. Read an integer score.
- Accept scores from 0 through 100 inclusive.
- A: 90-100, B: 80-89, C: 70-79, D: 60-69, F: 0-59.
- Report an invalid score outside that range.
- Test 59, 60, 89, 90, 100 and 101.

```
score = int(input("Score: "))
if not 0 <= score <= 100:
    print("Invalid score")
elif score >= 90:
    print("A")
elif score >= 80:
    print("B")
elif score >= 70:
    print("C")
elif score >= 60:
    print("D")
else:
    print("F")
```

Lab 3: Rock-Paper-Scissors Rules

- Create rps.py for one round against the computer.
- Accept rock, paper or scissors after `strip().lower()`.
- Equal choices produce a tie.
- Rock beats scissors. Scissors beats paper. Paper beats rock.
- Reject invalid input and print both valid choices.

```
import random

# choice() picks one item.
computer = random.choice(
    ["rock", "paper", "scissors"]
)

# First, test with fixed values:
player = "rock"
computer = "scissors"
# Expected outcome: You win
```

Lab 3: Branching Example

- Validate the player choice before deciding the outcome.
- Check for a tie before win and loss cases.
- List all three winning combinations.
- Try fixed choices to test every branch.

```
import random
p = input("Your choice: ").strip().lower()
options = ["rock", "paper", "scissors"]
c = random.choice(options)
if p not in ["rock", "paper", "scissors"]:
    print("Invalid choice")
else:
    print(f"You: {p}, computer: {c}")
    if p == c:
        print("Tie")
    elif ((p == "rock" and c == "scissors")
          or (p == "scissors" and c == "paper")
          or (p == "paper" and c == "rock")):
        print("You win")
    else:
        print("You lose")
```

Lab 4: Leap-Year Rules

- Create leap_year.py for positive integer years.
- Use the Gregorian leap-year rule.
- A year divisible by 400 is a leap year.
- Otherwise, a year divisible by 100 is not.
- Otherwise, a year divisible by 4 is a leap year. All other years are not.

```
# Divisibility uses remainder:  
print(2024 % 4 == 0)    # True  
print(1900 % 100 == 0)  # True  
print(2000 % 400 == 0)  # True
```

```
# Expected classifications:  
# 2024: Leap year  
# 1900: Not a leap year  
# 2000: Leap year  
# 2023: Not a leap year
```

Lab 4: Leap-Year Calculator

- Reject zero and negative years.
- Combine the rules with and and or.
- Parentheses separate the two ways a year can qualify.
- Print a clear result for the entered year.
- Challenge: express the same rules with if/elif/else.

```
year = int(input("Year: "))
if year <= 0:
    print("Enter a positive year")
else:
    is_leap = (year % 400 == 0 or
               (year % 4 == 0 and
                year % 100 != 0))
    if is_leap:
        print(f"{year}: Leap year")
    else:
        print(f"{year}: Not a leap year")
```

Testing & Submission

- Run each file and compare its output with the expected result.
- Check boundary values and invalid choices.
- For the game, test all nine valid player/computer pairs.
- Submit strings.py, grades.py, rps.py and leap_year.py.
- Add a comment showing at least three test inputs and their expected results per file.

String input	Expected
" "	No text entered
" A "	Length: 1
Score	Expected
59 / 60	F / D
100 / 101	A / Invalid score
Year	Expected
1900 / 2000	Not leap / Leap
2023 / 2024	Not leap / Leap
0	Positive-year message

Summary & Next Week

- Operators build expressions and comparisons.
- String indices start at 0. Slices exclude the stop index.
- String methods return results. Strings stay immutable.
- if/elif/else chooses the first matching branch.
- Boundary tests help reveal missing cases.

Week 3

Lists and tuples
Dictionaries and sets
for and while loops
range(), break and continue

Preview:
How could you repeat the game
until the player chooses to quit?



Questions?

Python Programming - Week 2